

RoomSpace: Constraint-Based Room Planning and Decoration Recommendation System

Project Proposal for UCS503P — Software Engineering Lab
Submitted to: Ms. Jhonsy Bansal
Thapar Institute of Engineering and Technology, Patiala

Pranjal Malhotra, CSED* Arnav Mittal, CSED†

13 August 2026

1 Higher-order goal

...secondary framing

RoomSpace aims to make practical interior planning accessible by turning room photographs, dimensions, and user requirements into feasible, visual layout alternatives. The immediate engineering objective is to translate *comfortable, usable room design* into a measurable and deployable software system rather than merely generating textual decoration suggestions.

2 Time-to-value

...primary heuristic

- We will first deliver the core room-model and 2D layout workflow: enter dimensions, define room elements, generate layouts, and inspect feasible alternatives.
- We will use short iterations to validate constraints and usability with representative room configurations before adding computer-vision assistance and product matching.
- CI-backed testing will protect the geometry, validation, and API layers as the system evolves.

3 Problem Statement

People planning or redecorating a room often find it difficult to convert a photograph, dimensions, and design preferences into a practical arrangement. Existing inspiration tools generally provide visual ideas but do not ensure that furniture fits, doors remain accessible, or circulation space is preserved. Key challenges include:

- **Unstructured input:** a room image and dimensions must be converted into a verified two-dimensional room representation.
- **Spatial feasibility:** furniture must remain within boundaries, avoid collisions, preserve door clearance, and maintain walking space.
- **Iterative design:** users need to add, move, or remove furniture without losing earlier alternatives.

*Roll No: 1024031040

†Roll No: 1024031044

- **Actionable recommendations:** suggested furniture and decor must suit the required footprint, category, theme, and budget, with direct product links.

Why this matters now (contextual relevance): A system that combines spatial constraints with an interactive floor plan can reduce trial-and-error in room planning and make recommendations more useful than generic inspiration alone.

4 Proposed Solution

...Software Proposition

4.1 Overview

Build a **web-based RoomSpace system** with the following components:

- **Room input and analysis:** users upload a photo or start with an empty room, then enter dimensions, purpose, occasion, theme, budget, and furniture requirements.
- **Verification workspace:** local pre-trained computer vision suggests visible objects; the user confirms object type, position, size, and whether it is fixed or movable.
- **Constraint-based layout engine:** generate candidate furniture arrangements, reject invalid layouts, and rank feasible alternatives.
- **Interactive 2D floor plan:** display room boundaries, doors, windows, existing furniture, and recommended items.
- **Refinement, history, and products:** interpret supported text commands, preserve layout versions, and match selected-layout requirements to catalogue products.

4.2 Core workflow

1. The user supplies a room photo or creates an empty-room plan, then enters dimensions and requirements.
2. The system constructs a room model containing boundaries, fixed elements, and verified furniture footprints.
3. The Layout Engine generates candidate arrangements and filters layouts that violate hard spatial constraints.
4. Feasible layouts are scored and displayed in a 2D plan; the user selects, refines, and saves a version.
5. The selected layout produces product recommendations that meet footprint, style, category, and budget requirements.

4.3 Operational constraints for an educational, tier-2/3 setting

- Keep the solution usable through responsive web design and a clear 2D interface.
- Avoid external LLM and inference API dependencies; use local pre-trained vision assistance and deterministic application logic.
- Limit natural-language interaction to supported room-design commands in the first release.

5 Solution Approach

...Engineering focus

This is a software engineering project centered on a structured room model, constraint-based spatial reasoning, an interactive 2D editor, versioned layouts, and product matching. Computer vision assists with identification; the application's layout engine makes the planning decisions.

5.1 Room analysis and verification

...assisted, user-controlled

A locally deployed pre-trained object detector may identify likely furniture and fixed room elements. Its output is always user-verifiable:

- Users correct object types, dimensions, positions, and fixed/movable status.
- Empty-room and furnished-room planning share the same Room Model and Layout Engine.
- Custom model training is not required for the initial version, and the system remains usable when vision assistance is unavailable.

5.2 Web architecture

...web-based solution

A typical 3-tier web architecture:

- **Frontend:** React with Vite, using SVG or Canvas for the interactive 2D room and furniture visualization.
- **Backend API:** FastAPI with Python for validation, layout orchestration, controlled command parsing, and product matching.
- **Spatial engine:** Python geometry algorithms for candidate generation, collision detection, door clearance, and ranking.
- **Database:** PostgreSQL or Supabase for rooms, objects, layouts, version history, and catalogue data.

5.3 Constraint logic and controlled refinement

Each furniture object has a position, width, depth, and rotation in a two-dimensional coordinate system. Doors and windows are fixed regions. Hard constraints ensure that furniture stays inside the room, does not overlap other items, leaves doors usable, avoids unnecessary window obstruction, respects fixed furniture, and maintains a configurable minimum circulation clearance. Feasible layouts are then ranked by accessibility, space quality, user preferences, theme, and budget.

The command window will use controlled NLP based on normalization, keywords, regular expressions, synonyms, and fuzzy matching. For example, “Add a table near the window” and “Move the desk to the left wall” are translated into structured operations and validated by the Layout Engine before execution.

5.4 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build the frontend and run backend unit and integration tests on every change.
- Run known-room-configuration tests for boundary, collision, clearance, and command-validation logic.
- Produce repeatable staging deployments for safe, incremental feature releases.

6 Evaluation Criterion

...measurable and attributable

We define success using measurable metrics tied to feasibility, correctness, and usability of the core workflow.

6.1 Primary evaluation metric

Valid-layout generation rate: proportion of generated candidate layouts that satisfy all hard spatial constraints after validation.

- **Target:** 100% of layouts presented as feasible must pass boundary, collision, fixed-object, door-clearance, and minimum-circulation checks.
- **Attribution:** measured through automated geometry tests and validation logs for representative room configurations.

6.2 Secondary evaluation metrics

- **Generation time:** time to produce and score a usable set of layout alternatives for a standard room configuration.
- **Command success rate:** proportion of supported refinement commands correctly parsed, validated, and applied.
- **User clarity:** user-reported clarity of the 2D plan and layout alternatives on a simple 1–5 feedback question.
- **Recommendation fit:** proportion of displayed products that meet the selected layout’s required footprint, category, and budget filters.

6.3 Pilot validation plan

...high-level

- Test with furnished and empty-room scenarios of different sizes, including door and window constraints.
- Ask pilot users to complete planning and refinement tasks, then collect task outcomes and usability feedback.
- Compare engine output against expected valid and invalid configurations recorded in a test suite.

7 Scalability

...with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in room plans, stored versions, and concurrent users by:
 - decoupling components (frontend/backend),
 - indexing commonly accessed room, layout, and product records,
 - using stateless API design.
2. **Operationally deployable (practically):** The system should run on common web hosting with:
 - a managed PostgreSQL-compatible database,

- containerized or platform-hosted deployment,
- CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- React, Vite, and SVG/Canvas for a responsive interactive interface.
- FastAPI and Python geometry libraries for API, validation, and layout processing.
- A locally deployed pre-trained object detector for optional room-object assistance.
- PostgreSQL or Supabase for persistent room, layout-version, and product-catalogue data.
- Test frameworks, a CI runner, and a staging deployment target.

We will use these mature components to focus on workflow design, correctness, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables

...iteration-friendly

9.1 Initial deliverable

...first iteration

- Room input form for dimensions, purpose, theme, budget, and furniture requirements
- Interactive 2D room model with boundaries, furniture footprints, doors, and windows
- Deterministic candidate generation with boundary, collision, and clearance validation
- Layout scoring, ranking, and persistent version history
- CI: frontend build, backend unit tests, geometry tests, and linting
- Deployment to staging through a repeatable pipeline job

9.2 Subsequent deliverables

- Local computer-vision assistance with a user verification step
- Controlled text commands for adding, moving, removing, and fixing furniture
- Product catalogue filtering, ranking, budget checks, and direct product URLs
- Improved search, indexing, feedback capture, and visualization enhancements

10 Risks and mitigations

- **Incorrect vision suggestions:** keep vision assistance optional and require user verification before planning.
- **Invalid layouts:** enforce hard constraints in a testable geometry engine before rendering any recommendation.
- **Ambiguous user commands:** support a bounded command grammar and show validation feedback rather than guessing.

- **Scope growth:** prioritize the 2D room model and constraint engine; treat photorealistic visualization as an optional future enhancement.
- **Product-data quality:** begin with a structured application catalogue and validate dimensions, budget, and URLs.

11 Summary

RoomSpace proposes a deployable web platform for generating practical room-layout and decoration recommendations. It combines a verified room model, constraint-based layout engine, interactive 2D visualization, controlled refinement commands, version history, and product matching. The project emphasizes rapid, testable engineering increments, CI/CD-backed delivery, and measurable spatial-validity and usability criteria rather than dependence on external LLM services or research novelty.